LoRazor: Trimming Adapter Execution for Efficient Multi-LoRA Serving

Xiang Li Pengcheng Wang Azam Ikram Saurabh Bagchi

Purdue University

arXiv:2611.10049v1 [cs.AI] 5 Nov 2026

Abstract

LoRA offers a parameter-efficient adaptation strategy, supporting multi-user workloads via customized adapters on a shared backbone. However, serving concurrent requests with different adapters introduces substantial computation and memory-management overhead. We ask the research question: Do all requests need their adapters throughout the whole generation process? Our motivational study reveals that employing LoRA adapters at every decoding step is unnecessary; releasing them early has limited impact on accuracy while substantially accelerating generation. Although skipping adapters early offers theoretical workload reduction, translating these computational savings into concrete throughput gains requires tailored GPU kernels. We present LoRazor, a multi-LoRA serving framework that combines early adapter release with active-row-aware GPU kernels. LoRazor determines when to permanently release a request's adapter and continue decoding with the backbone alone. By dynamically resizing LoRA kernel launches as adapters are released, LoRazor trims idle GPU blocks. Across two backbones and four tasks, LoRazor achieves up to 3.02× request throughput and 2.53× output-token throughput over vLLM multi-LoRA, with a mean relative task-accuracy decrease of 2.89%.

1 Introduction

Large language models (LLMs) support a broad range of tasks, from answering questions to generating code. However, their general-purpose capabilities may fall short when applications require specialized domain knowledge, task-specific reasoning, or precise output formats. Low-rank adaptation (LoRA) (Hu et al., 2022) provides a parameter-efficient way to specialize these models by freezing the pretrained backbone and learning low-rank, task-specific weight updates (Kong et al., 2024; Zhang et al., 2024; Li et al., 2026d). A provider (e.g., HuggingFace) can then serve many such adapters on a shared backbone rather than maintain a full model copy for each task. Despite the compact size of individual adapters making it possible to fit more adapters in GPU memory, serving those adapters concurrently still leads to resource contention, which degrades serving performance.

Prior work addresses this through efficient multi-adapter GPU execution and memory management (Chen et al., 2024; Sheng et al., 2024; Wu et al., 2024a), shared adapter representations (Kopiczko et al., 2024; Gabrielsson et al., 2025; Li et al., 2026c), and prompt-only adaptation (Ma et al., 2025; Lanpouthakoun et al., 2026). We examine a complementary question: how long does each request need its assigned adapter during the decoding stage? Specifically, can the request stop using its adapter before generation finishes and use the backbone alone to complete the response? We refer to this permanent adapter deactivation for a specific request as *adapter release* and investigate its quality-efficiency trade-off.

We conduct a comprehensive motivational study on four datasets and two backbones. Our experiments (Sec 2) show that the impact of adapter release on task accuracy varies across tasks and backbones. Under certain circumstances, applying the adapter during prefill and several initial decoding iterations already preserves much of the adapted model's accuracy. In others, early release may substantially degrade quality. These observations motivate our selective adapter release but also raise two questions: *When should each request stop using its adapter, and how can the serving system translate these decisions into higher serving throughput?*

Deciding when to release an adapter requires assessing its effect on the remaining generation. Because adapter release changes both subsequent token generation and the KV cache, its impact on final task quality may not be apparent immediately. Thus, a practical release policy must account for the downstream effects while adding little overhead to the batch execution time. Furthermore, adapter release alone does not ensure higher serving efficiency. As requests release their adapters, fewer tokens need to compute LoRA updates in each batch, but the existing LoRA computation kernel (Kwon et al., 2023) remains sized for

the full padded batch, leaving excess GPU blocks that incur block-scheduling and metadata-access overhead without performing useful matrix computation. Realizing the benefits of adapter release therefore requires adapting kernel launches to the adapter workload at every iteration.

To solve the aforementioned problems, *we present LoRazor, a multi-LoRA serving framework that combines selective adapter release with workload-aware kernel execution.* LoRazor consists of two components: StepTrim and GridTrim. StepTrim (Sec 3.2) uses a release policy to decide when a request releases its adapter and continues decoding with the backbone alone. This reduces the number of tokens processed by each adapter, creating opportunities to shrink the corresponding GPU launch grids. RowTrim (Sec 3.1) exploits these opportunities by adapting grouped LoRA kernel launches to the remaining adapter workload. Together, they reduce adapter computation and translate reductions in active LoRA group sizes into kernel execution savings.

We implement LoRazor in vLLM (Kwon et al., 2023) and evaluate it on two widely used backbone LLMs, Llama-3.2-3B and Mistral-7B, with adapters spanning four tasks from four fields including mathematics, coding, medical and science. LoRazor achieves throughput speedups of 2.56× and 3.02×, respectively, over vLLM multi-LoRA serving (Punica derived LoRA kernels), with an average accuracy decrease of 1.82 percentage points, which is only 2.89% relative to the full-LoRA execution. Our contributions are:

- We identify and systematically exploit opportunities to reduce multi-LoRA decoding cost through selective adapter release and reduction of inactive GPU blocks.
- We design and develop LoRazor, which consists of StepTrim for early selective adapter release and RowTrim for adapting grouped LoRA kernel launches to active adapters.
- We integrate LoRazor into the state-of-the-art vLLM serving engine and evaluate its quality-throughput trade-offs using diverse tasks and backbones to demonstrate its efficiency.

2 Motivation

2.1 Adapter Diversity Increases Decode Latency

A multi-LoRA batch contains requests that need to be assigned to different adapters. Existing framework (Kwon et al., 2023) provision GPU work using the full batch-wide row dimension for each active adapter group, even when an adapter is assigned only a small subset of requests. Consequently, higher adapter diversity creates additional GPU work that is not proportional to the number of requests that are tied to each adapter. Figure 1(a) demonstrates this effect for a batch of 256 requests. Increasing the number of distinct active adapters from 8 to 128 raises the median per-step decode latency from 21.91 ms to 48.82 ms for Llama-3.2-3B and from 35.72 ms to 80.04 ms for Mistral-7B-v0.3.

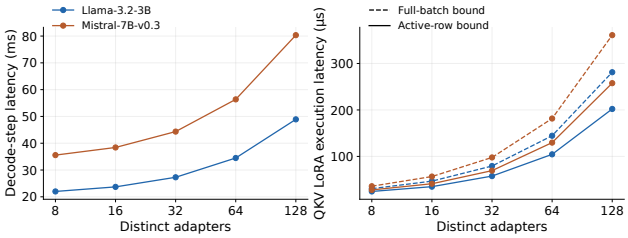

Figure 1. (a) For a fixed batch of 256 requests, per-step latency increases with the number of distinct adapters. (b) Using active row bound reduces the LoRA execution latency.

Figure 1(b) shows the cost of oversized kernel launches. The full-batch bound provisions 256 rows for every adapter regardless of how many rows are actually being used. We instead use an *active-row bound*: a shared row bound determined by the largest number of tokens assigned to any active adapter, rounded up to a supported launch size. This reduces the row tiles provisioned for each adapter while retaining all required adapter computation. We compare LoRA execution latency when using these two launch bounds. At 128 adapters, the active-row bound reduces latency by 28.1% on Llama-3.2-3B and 28.6% on Mistral-7B-v0.3. These results demonstrate the cost of provisioning row tiles beyond the active adapter workload and motivate *GridTrim*, which adapts kernel launch bounds to active adapter-group sizes.

2.2 LoRA Is Not Required at Every Decode Step

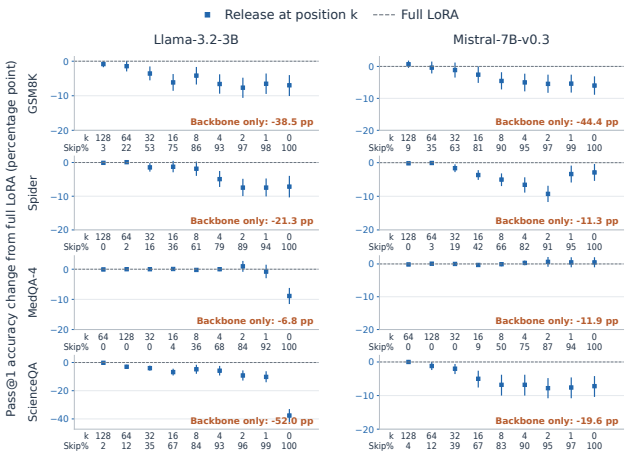

Figure 2. The quality impact of decode-time LoRA adapter release varies across tasks and backbones. The best release position is task- and backbone-dependent.

Existing system applies a request's adapter during prefill

and every subsequent decode step. However, the adapter's contribution is not equally important throughout the generation. We study this behavior by disabling LoRA during decoding steps while retaining it throughout prefill.

Figure 2 evaluates adapter releases at fixed positions. The adapter remains enabled during prefill and the first k decoding steps, after which it is disabled for the remainder of generation. Skip (%) denotes the fraction of decoding steps that omit adapter, relative to applying an adapter throughout decoding. We report test-accuracy changes relative to using an adapter throughout generation and the mean of fraction of decode adapter calls skipped across requests. The quality-efficiency trade-off varies substantially across tasks and backbones. On MedQA-4, releasing after one decode step skips 92.0% of decode adapter calls on average for Llama-3.2-3B while changing accuracy by -0.63 percentage points. In contrast, early release causes larger accuracy losses on GSM8K and ScienceQA tasks. Therefore, a fixed release point does not preserve quality uniformly. Furthermore, retaining LoRA for more decoding steps also does not always improve accuracy. Mistral-Spider exhibits a nonmonotonic trend: accuracy decreases from $k = 0$ to $k = 2$, then generally recovers as release is delayed. A close examination suggests that the first few adapter-guided steps can produce SQL prefixes that the backbone struggles to complete correctly after release. Releasing immediately after prefill instead lets the backbone generate both the initial SQL tokens and their continuation. Delaying release further generally improves accuracy. Our study in Appendix A.1 shows disabling LoRA during prefill causes substantially larger accuracy losses, so we retain adapters throughout prefill and focus on release during decoding.

Together, we identify two opportunities to improve multi-LoRA serving efficiency: avoiding unnecessary GPU blocks for active adapters and shortening adapter execution when task accuracy permits. These findings motivate LoRazor, where StepTrim decides when to permanently release an adapter and GridTrim resizes grouped LoRA kernel launches according to the remaining active adapter rows.

3 Method

3.1 GridTrim: Active-Row LoRA Execution

GridTrim matches LoRA kernel launches to active adapter group sizes rather than the full batch. It uses a shared active-row bound for shrink and optionally separates expand into small- and large-class launches.

3.1.1 Identifying Redundant GPU Blocks

Let $X \in \mathbb{R}^{M\times H}$ contain a linear layer's input activations, where M is the padded batch row count and H is the input hidden dimension. Adapter a serves row indices $\mathcal{I}_a$, with $m_a = |\mathcal{I}_a|$, and $\mathcal{A} = \{a : m_a > 0\}$ denotes the active adapters. For LoRA weights $A_a \in \mathbb{R}^{H\times r_a}$ and $B_a \in \mathbb{R}^{r_a\times D}$, the adapter computes

$$U_a = X[\mathcal{I}_a]A_a, \qquad \text{(shrink)} \qquad (1)$$

$$\Delta Y_a = U_a B_a, \qquad \text{(expand)} \qquad (2)$$

where r_a is the adapter rank and D is the output width. The correction $s_a\Delta Y_a$ is added to the backbone output at rows $\mathcal{I}_a$, with adapter scaling factor s_a.

For row-tile height B_M, the baseline (Kwon et al., 2023) launches $\lceil M/B_M\rceil$ row tiles per adapter slot, although only $\lceil m_a/B_M\rceil$ tiles perform matrix computation. Remaining tiles read the group's row count and exit, still incurring scheduling and metadata-access overhead (Figure 3 right). For example, with $M = 256$ and $B_M = 16$, an adapter serving 10 rows needs one row tile but receives sixteen.

Let L be the number of adapter slots provisioned by the launch. The baseline provisions $L\lceil M/B_M\rceil$ row tiles. GrdiTrim aims to reduce this by provisioning GPU kernel launches according to the active adapter request assignment in each batch.

3.1.2 Specializing the Row Bound

The full batch size is a conservative bound on every adapter's workload. A tighter bound is the largest active adapter group, $m_{\max} = \max_{a\in\mathcal{A}} m_a$, in the current batch. GridTrim selects a common row bound R satisfying $m_{\max} \le R \le M$ and substitutes it for M in the row dimension of both shrink and expand launch grids, resulting in $L\lceil R/B_M\rceil$ row tiles. The bound only changes the launch geometry. Blocks continue to use the existing adapter-grouped row-index array to gather input activations and scatter results to their original batch positions.

The launch additionally spans S projection slices, L adapter slots and P split-K partitions. Equation 3 gives the resulting three-dimensional launch grid:

$$G_R = \left(P\left\lceil\frac{R}{B_M}\right\rceil\left\lceil\frac{N}{B_N}\right\rceil,\ S,\ L\right) \qquad (3)$$

For shrink, N is the LoRA rank; for expand, $N = D$ and $P = 1$ since the expand kernel does not partition its reduction over the LoRA rank across blocks. We denote their stage-specific grids by G_R^{shrink} and G_R^{expand}, respectively. On the shared-bound path, both use the same R and L, with stage-specific tile settings. Row-bound specialization replaces M with R in the launch grid and its block-coordinate calculation, leaving the other grid dimensions and the underlying computation unchanged.

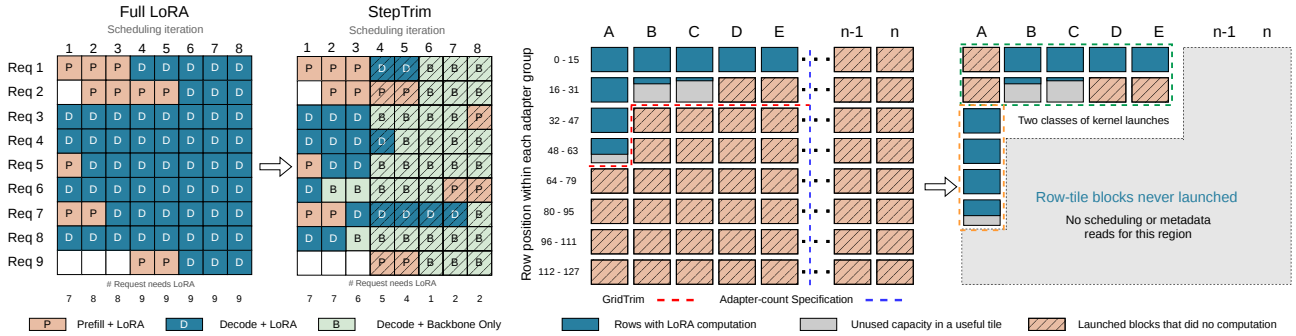

Figure 3. Overview of LoRazor. Left: StepTrim retains LoRA during prefill and releases adapters during decoding. Requests then continue with the backbone alone and may terminate at different steps from full-LoRA execution. Right: GridTrim reduces the row dimension of grouped LoRA launch grids, complementing vLLM's adapter-count specialization. Two-class expand execution handles imbalanced adapter-group sizes.

3.1.3 Separating Row-Size Classes

A shared row bound allows one large adapter group to enlarge every adapter's launch grid. This happens frequently when a batch contains both prefill and decode requests. To address this, we partition active adapters into two classes:

$$\mathcal{A}_s = \{a : 0 < m_a \le 16\}, \qquad \mathcal{A}_l = \{a : m_a > 16\} \quad (4)$$

The small class uses row bound $R_s = 16$, while the large class uses $R_l = R$, where R covers the largest active group. These bounds specify row coverage, not tile height. Shrink retains its shared-bound launch. We only apply class separation to expand as expand tiles span a much wider output dimension rather than the LoRA rank, so redundant row tiles are replicated across more output-column tiles, making expand a natural target for class separation.

To support fixed launch shapes for CUDA-graph replay, the small class retains the original L adapter slots, marking nonmembers inactive. The large class uses a compact metadata list with a fixed capacity of $C = 16$ slots. Each slot identifies an adapter. Unused slots exit without matrix computation. For class c with row bound R_c and adapter-slot capacity L_c, the expand launch grid is

$$G_c^{\text{expand}} = \left(\left\lceil \frac{R_c}{B_M} \right\rceil \left\lceil \frac{D}{B_N} \right\rceil, \; S, \; L_c \right) \quad (5)$$

where D is the projection's output width, S is the number of projection slices, and B_M and B_N are the row- and column-tile sizes. With identical tile sizes, the shared-bound and separating class paths provision

$$T_{\text{shared}} = L \left\lceil \frac{R}{B_M} \right\rceil, \; T_{\text{class}} = L \left\lceil \frac{16}{B_M} \right\rceil + C \left\lceil \frac{R}{B_M} \right\rceil \quad (6)$$

Class separation targets batches containing many small adapter groups and a few large ones, as can arise when decode requests share a batch with prefill requests. A shared bound lets the larger prefill groups inflate the expand grids of smaller decode groups. Separating them reduces this over-provisioning, but adds a kernel launch and metadata preparation. We therefore enable class-separated expand only when $|\mathcal{A}_s| \ge 8$ and $1 \le |\mathcal{A}_l| \le C$. The small-group threshold heuristically targets batches with sufficient opportunity to optimize the batch performance. Other batches, including capacity overflow, use the shared-bound path without dropping any adapter work.

3.1.4 Selecting and Replaying Row-Bucket Graphs

Adapting the row bound also requires accommodating CUDA graphs. vLLM captures GPU operations for a fixed execution shape and replays that sequence on subsequent decoding steps to reduce CPU launch overhead. Because a captured graph fixes its kernel launch dimensions, a single graph cannot directly accommodate arbitrary values of R. We therefore capture a small set of row-bound and class-layout variants. The configured class policy is fixed during engine initialization. For padded batch size M, the supported bounds are

$$\mathcal{R}(M) = \{16 \cdot 2^k \mid k \in \mathbb{Z}_{\ge 0}, \; 16 \cdot 2^k < M\} \cup \{M\} \quad (7)$$

For example, a batch with $M = 256$ uses bounds of 16, 32, 64, 128, or 256. Each supported bound corresponds to a pre-captured graph. Let d denote the existing graph descriptor, including the padded batch shape and adapter-count configuration. We extend it with R and a layout flag z: $z = 0$ selects shared-bound execution, and $z = 1$ selects class-separated expand. The corresponding graph is $\Gamma(d, R, z)$. At each decoding iteration, the host computes the assigned token-row count m_a for each adapter by summing the scheduled token counts of requests using that adapter. For a nonempty active adapter set, we select the smallest supported bound covering the current $m_{\max}$:

$$R = \min\{r \in \mathcal{R}(M) : r \ge m_{\max}\} \quad (8)$$

Algorithm 1 GRIDTRIM graph selection and execution.

Require: Graph descriptor d with padded row count M, active adapter set $\mathcal{A}$, row lists $\{\mathcal{I}_a\}_{a\in\mathcal{A}}$
Runtime graph selection
if $\mathcal{A} = \varnothing$ **then**
  Execute the backbone-only path
  return
end if
for $a \in \mathcal{A}$ **do**
  $m_a \leftarrow |\mathcal{I}_a|$
end for
$m_{\max} \leftarrow \max_{a\in\mathcal{A}} m_a$
$R \leftarrow \min\{r \in \mathcal{R}(M) : r \geq m_{\max}\}$
$z \leftarrow$ whether the class eligibility rule holds
Prepare adapter metadata, including class lists if enabled
Replay $\Gamma(d, R, z)$ with current inputs and metadata
Within replay, for each LoRA operation:
Run shrink with captured grid G_R^{shrink} to compute the intermediate activations
if $z = 1$ **then**
  for each configured class c **do**
    Run expand with captured grid G_c^{expand}
  end for
else
  Run expand with captured grid G_R^{expand}
end if

The host selects $z = 1$ when the class-separation conditions in Section 3.1.3 hold, and $z = 0$ otherwise. The engine then replays $\Gamma(d, R, z)$ with current inputs and adapter metadata. Within replay, shrink and the selected expand launches execute in dependency order using their captured grids. Selecting a different bucket or layout therefore changes the launch shape without recapturing the graph; if the bucket and existing descriptor remain unchanged, the same graph is reused. If a valid row-bound descriptor is unavailable, the kernel retains the full-batch bound M. Algorithm 1 summarizes this process.

3.2 StepTrim: Decode-Time adapter release

Applying a LoRA adapter at every decoding step is unnecessary for some requests, but deciding when to release the adapter of a request is difficult. We propose STEPTRIM to determine when to permanently release a request's adapter and continue decoding with the backbone alone. STEPTRIM supports two release policies: a profile-based training-free policy that uses an adapter-task specific fixed release length k, and a learned policy that makes request-specific decisions. Both policies retain LoRA adapter during prefill and preserve the existing KV cache at release. The profiled-based policy requires no training. It calibrates a Pareto-frontier of the release position k and task accuracy on the validation dataset, and picks the position k that matches the serving requirements. The learned policy exploits differences in release sensitivity among requests. Our evaluation shows its additional benefits are task-backbone dependent.

3.2.1 Profiled Release Policy

The profiled-based policy selects a fixed release length k for each task-adapter pair. We evaluate candidate release lengths and their corresponding accuracies on the validation set, retaining LoRA during prefill and the first k decoding steps before switching to backbone-only decoding. The existing KV cache is preserved at release. $k = 0$ corresponds to prefill-only execution. We select k according to the desired validation quality-efficiency criterion and freeze it before deployment. All requests assigned to the same adapter use the same selected release length. This policy, used by LoRazor-Training-free, requires offline profiling but no training.

3.2.2 Learned Release Policy

One-way adapter release Consider request i, assigned to adapter a_i. Let $z_{i,t} \in \{0, 1\}$ denote whether its adapter is active at decode step t. $z_{i,0} = 1$ as we always apply the adapter during prefill. After a decode forward pass, it reads the final hidden vector $h_{i,t} \in \mathbb{R}^H$. Then we apply a small trained predictor associated with the request's adapter to compute

$$s_{i,t} = w_{a_i}^{\mathsf{T}} h_{i,t} + b_{a_i} \tag{9}$$

The adapter assignment for the next decode step is

$$z_{i,t+1} = z_{i,t}\, \mathbf{1}[s_{i,t} \geq \tau_{a_i}] \tag{10}$$

where τ_{a_i} is a calibration threshold selected on validation data. Once $z_{i,t}$ becomes zero, it remains zero until the request terminates. The request then follows the backbone-only path, while other requests in the same batch may continue using their adapters.

Labeling with paired continuations The desired training label is not whether LoRA changes the next token, but whether releasing it early harms the final task outcome. For each training request, we apply the corresponding LoRA adapter normally and sample reachable decode positions. Those positions can be changed based on the dataset's generation length. For the dataset used in our evaluation, we sample at the following positions.

$$\mathcal{T} = \{0, 1, 2, 3, 7, 15, 31, 63\} \tag{11}$$

We also tried to increase the number of sampling points at different positions along with more complex architectures, but the improvement is marginal.

At each position t, we fork the generation state into two free-running continuations:

ON: retain the adapter from t through EOS;

OFF: disable the adapter from t through EOS.

Both begin from an identical state. The OFF branch generates tokens until termination without using an adapter. Consequently, its final outcome captures both the immediate effect of the decision and its downstream effect through changed tokens and attention states.

Let $Q(\cdot) \in \{0, 1\}$ be the task evaluator, such as answer accuracy or SQL execution correctness. We label a candidate release point as risky if continuing with LoRA produces a correct answer, but disabling LoRA from that point onward produces an incorrect answer:

$$y_{i,t} = \mathbf{1}[Q(Y_{i,t}^{\mathrm{ON}}) = 1 \ \wedge Q(Y_{i,t}^{\mathrm{OFF}}) = 0] \qquad (12)$$

Thus, the predictor learns to rank the risk of releasing an adapter at a position. The training label does not treat a release as harmful when both branches fail.

Lightweight risk-predictor training We train one linear predictor per adapter using only its training split. Because longer generations can contribute more sampled decoding points, averaging over all points would overweight such requests. We instead give every request equal total weight:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{|\mathcal{T}_i|} \sum_{t \in \mathcal{T}_i} \mathcal{L}_{\mathrm{BCE}}\big(w^{\mathsf{T}} h_{i,t} + b,\ y_{i,t}\big) + \frac{\lambda}{2} \|w\|_2^2 \qquad (13)$$

Here, N is the number of training requests, $\mathcal{T}_i$ contains the sampled decoding points reached by request i, and $\mathcal{L}_{\mathrm{BCE}}$ denotes binary cross-entropy between the sigmoid-transformed logit and the risk label. The final term regularizes the head weights, with $\lambda = 0.1$ across all adapters and training-set sizes.

The trained predictor produces a risk score $s_{i,t} = w_{a_i}^{\mathsf{T}} h_{i,t} + b_{a_i}$. We permanently release request i's adapter when $s_{i,t} < \tau_{a_i}$, where τ_{a_i} is an adapter-specific threshold. Larger thresholds permit more aggressive release. We select τ_{a_i} using validation data for calibration. Candidate thresholds are drawn from quantiles of the predictor's training scores. For each candidate, we generate complete responses on the validation split and measure accuracy loss relative to full LoRA and the fraction of decode adapter calls skipped. We retain the Pareto-optimal candidates and select an operating point according to the desired quality-efficiency trade-off, freezing the threshold before deployment.

4 Evaluation

4.1 Models and Datasets

We evaluate Llama-3.2-3B and Mistral-7B-v0.3 on mathematical reasoning (GSM8K (Cobbe et al., 2021)), text-to-SQL generation (Spider (Yu et al., 2018)), medical question answering (MedQA-4 (Jin et al., 2021)), and scientific question answering (ScienceQA (Lu et al., 2022)), training one LoRA adapter per backbone-task pair. Table 2 summarizes the datasets. GSM8K and Spider reserves training examples for validation and retain the official test split. MedQA-4 uses four-option questions; ScienceQA uses text-only examples from the official splits. We report pass@1 accuracy from greedy generation, scored by numerical answer correctness for GSM8K, SQL execution correctness for Spider, and answer-choice correctness for MedQA-4 and ScienceQA. Output limits are 256 tokens for GSM8K and Spider, 128 for MedQA-4, and 320 for ScienceQA.

Table 2. Processed dataset sizes.

Dataset	Training	Validation	Test
GSM8K	6726	373	1319
Spider	6282	359	1034
MedQA-4	10178	636	1273
ScienceQA	5000	250	500

4.2 Implementation and Serving Setup

We implement LoRazor on vLLM v0.30.0 (released on Sept 22, 2026) and conduct all experiments on NVIDIA H100 80GB GPUs. Throughput workloads contain 32768 requests, equally divided among four tasks, with test examples sampled repeatedly as needed. To vary adapter diversity from 4 to 128, we replicate the four trained task adapters under distinct logical identities. At 128 adapters, each task adapter has 32 identities with identical weights but separate GPU allocations, each executed as a distinct adapter. Thus, weights are not shared across identities. Main experiments assign requests uniformly across identities, with skewed traffic evaluated separately (Sec 5.2). The learned StepTrim predictor is trained on 128 requests per task–backbone pair. For LoRazor-P25/P50/P75, we select thresholds τ that maximize validation accuracy while achieving at least 25/50/75% decode skipping, respectively, and freeze them before testing. Realized test-time skipping may differ. Additional serving details appear in Appendix A.2.

4.3 Baselines

For task accuracy, we compare LoRazor against vLLM, S-LoRA, Prefill-only, and PreFT (Lanpouthakoun et al., 2026). vLLM/S-LoRA applies LoRA during prefill and every decoding step. Prefill-only applies LoRA only during prefill. PreFT is similar to Prefill-only, but it uses specialized training so the adapter is trained to only used on prefill but not decode. We also compare against our own LoRazor-Training-free (without GridTrim), which disables LoRA after the first k decoding steps. The selection of k is based on the validation dataset of each backbone-task pair, details can be found in Appendix A.3. For throughput, we compare against vLLM 0.30.0's default multi-LoRA serving, which incorporates Punica-derived kernels for heterogeneous adapter batching and CUDA-graph execution, S-

Table 1. Test-split Pass@1 accuracy (%). Full LoRA applies LoRA throughout prefill and decoding. LoRazor-Training-free uses the profiled threshold k on each validation split that gives at least 50% calibration-time decode skipping. For LoRazor-P25/P50/P75, each adapter's threshold τ_a is selected independently on the validation split to achieve at least 25%/50%/75% calibration-time decode skipping, respectively. LoRazor-P50 is used in the throughput test unless specified.

	Llama-3.2-3B				Mistral-7B-v0.3			
Method	ScienceQA	Spider	MedQA-4	GSM8K	ScienceQA	Spider	MedQA-4	GSM8K
Backbone-only	38.00	44.00	45.48	6.14	74.00	57.06	47.45	10.54
PreFT (Lanpouthakoun et al., 2026)	82.60	59.96	50.35	35.86	87.40	64.60	59.23	49.20
Prefill-Only	52.60	58.25	45.95	37.55	85.93	65.22	60.02	48.02
LoRazor-Training-free	83.20	63.31	52.37	41.55	88.33	64.44	59.60	48.02
LoRazor-P25	88.40	63.54	52.50	41.58	92.07	65.76	59.99	53.65
LoRazor-P50	87.27	62.54	52.42	41.24	90.73	65.76	59.99	52.59
LoRazor-P75	84.13	61.51	52.74	39.68	87.00	63.19	59.32	50.72
vLLM (Full LoRA) (Kwon et al., 2023)	90.00	65.34	52.55	44.02	92.93	68.44	59.47	54.33
S-LoRA (Sheng et al., 2024)	90.00	64.99	52.55	44.35	92.80	68.57	59.70	55.12

LoRA, and LoRazor-Training-free. We exclude prefill-only execution and PreFT from throughput comparisons because they incur substantial accuracy losses on some tasks and offer no flexibility for accuracy-throughput trade-off.

4.4 Downstream Task Performance

Table 1 reports test-split Pass@1 accuracy across four tasks and two backbones. At the P50 operating point used in the main throughput evaluation, LoRazor remains within 2.80 percentage points of full-LoRA accuracy on every backbone–task pair. It achieves higher accuracy than prefill-only execution on seven of eight pairs and PreFT on all eight. Compared with LoRazor-Training-free, LoRazor-P50 achieves higher accuracy on six of eight pairs, including all four Mistral tasks. Both policies are calibrated to achieve at least 50% validation-time decode skipping, using an adapter-specific release length for the training-free policy and a predictor threshold for the learned policy. Their realized test-time skipping rates can differ. Appendices A.3 and A.4 provide the selected release lengths and P50 test-time skipping rates, respectively. Table 4 further compares the policies under a common validation accuracy loss budget. Increasing the validation-time skipping target from P25 to P75 generally reduces accuracy, providing adjustable operating points for the quality-efficiency trade-off.

4.5 Throughput Improvement

Throughput experiments use the validation-selected P50 operating point unless otherwise specified. We evaluate both offline serving, where all requests are available at the start, and open-loop serving, where requests arrive according to a Poisson process. We construct workloads from the test splits of the four tasks and assign requests to 4-128 logical adapters. Under uniform traffic, each adapter receives an equal share of requests over the workload, although individual runtime batches may be imbalanced.

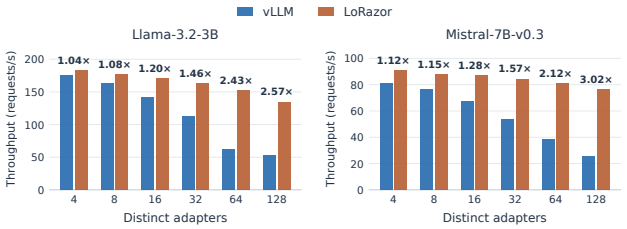

Figure 4. Throughput versus adapter diversity under uniform traffic. Labels show LoRazor's speedup over vLLM.

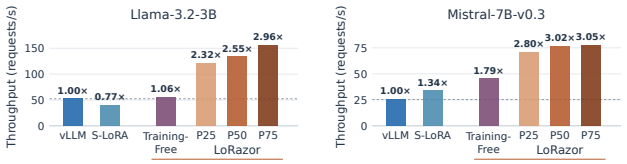

Figure 5. Throughput with 128 adapters under uniform traffic. Labels show speedup over vLLM. Training-free uses default vLLM kernels; P25/P50/P75 use GridTrim.

Offline throughput. Figure 4 shows that vLLM's throughput decreases as adapter diversity increases, consistent with the rising per-step decoding latency in Figure 1. LoRazor reduces this latency by selectively disabling adapter execution and sizing the remaining LoRA kernel launches to the active adapter workload. As adapter diversity increases from 4 to 128, its throughput speedup over vLLM grows from 1.04× to 2.57× on Llama-3.2-3B and from 1.12× to 3.02× on Mistral-7B-v0.3. Figure 5 compares the systems at 128 logical adapters. LoRazor achieves

Figure 6. Open-loop serving on Llama-3.2-3B under Poisson arrivals with 128 adapters under uniform traffic: achieved throughput (left) and P90 TTFT and TPOT (right).

Table 3. Output statistics for throughput experiments in Figure 5.

	Llama-3.2-3B		Mistral-7B-v0.3	
Method	10^3 tok/s	Tok/req	10^3 tok/s	Tok/req
vLLM	2.79	53.33	1.65	65.31
S-LoRA	2.14	53.23	2.21	65.33
LoRazor-Training-free	2.72	48.99	2.51	55.16
LoRazor-P25	6.23	51.23	4.20	59.26
LoRazor-P50	6.71	50.19	4.18	54.65
LoRazor-P75	7.77	50.02	4.14	53.55

2.55× and 3.02× vLLM's request throughput on Llama and Mistral, respectively, compared with 1.06× and 1.79× for Training-free (without GridTrim), and 0.77× and 1.34× for the S-LoRA compatibility port. Although LoRazor-P50 produces slightly shorter responses on average, its output-token throughput also increases by 2.40× and 2.53× over vLLM, respectively (Table 3). To isolate release-policy benefits, we additionally compare profiled-based (Training-free) and learned policy with GridTrim enabled for both under the same 2-percentage-point validation loss budget in Sec 5.1.2.

Open-loop serving. Figure 6 evaluates Llama-3.2-3B under Poisson arrivals ranging from 25 to 150 requests/s with uniform adapter assignment. We report throughput and the P90 time to first token (TTFT) and time per output token (TPOT). LoRazor closely tracks the arrival rate through 125 requests/s, whereas vLLM, the LoRazor-Training-free policy (without GridTrim), and S-LoRA plateau at approximately 51, 59, and 39 requests/s, respectively. At 100 requests/s, LoRazor maintains a P90 TTFT of approximately 55 ms and P90 TPOT of 25 ms, while vLLM experiences substantial queueing and a P90 TPOT near 99 ms. At 150 requests/s, LoRazor throughput plateaus near 126 requests/s and its P90 TTFT rises sharply, indicating overload. These results show that the offline throughput gains translate into a higher supported arrival rate before substantial queueing occurs.

4.6 StepTrim and GridTrim Overhead

Figure 7 reports StepTrim's release-decision latency and LoRA execution latency under single-class and two-class GridTrim. For StepTrim, we measure the learned policy's

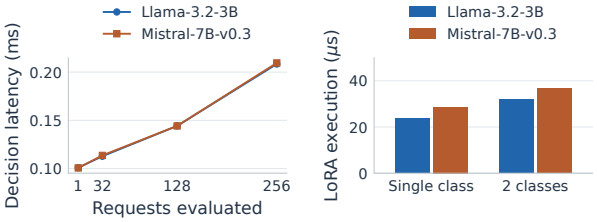

Figure 7. Left: StepTrim's 95th-percentile end-to-end release-decision latency with 128 predictor heads. Right: LoRA execution latency with single-class and two-class GridTrim.

decision overhead. The profiled policy only checks whether each request has reached its preselected release step and requires no predictor evaluation. The StepTrim measurement includes hidden-state gathering, predictor evaluation, decision transfer, and release-state updates. We keep 128 adapter-specific predictor heads resident and vary the number of evaluated requests from 1 to 256; each request uses only its assigned adapter's head, which may be shared by multiple requests. The 95th-percentile decision latency increases from approximately 0.10 ms to 0.21 ms, about 1% of a 20 ms decode step. Two-class execution adds metadata preparation and an additional expand launch, but the measured overhead is on the microsecond scale. Aside from latency overhead, GridTrim's additional row-bound and class-layout graph variants increase GPU memory consumption during capture by 1.61 GB relative to vLLM in our Llama-3.2-3B configuration. These graphs are captured during initialization and reused during decoding. Appendix A.5 evaluates the offline cost of collecting paired continuations and fitting the predictor, along with the effect of training-set size on task accuracy.

5 Ablation

5.1 Component ablation

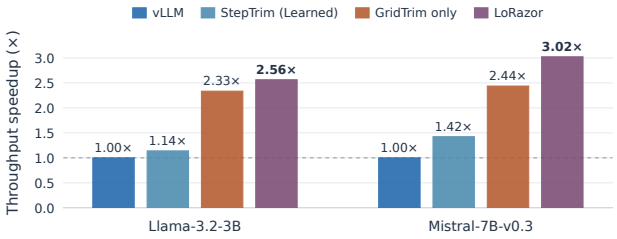

Figure 8. Component ablation under uniform traffic with 128 logical adapters. StepTrim and GridTrim both contribute to the improved performance.

Figure 8 isolates the contributions of StepTrim and GridTrim under uniform traffic with 128 logical adapters. GridTrim alone achieves throughput improvements of 2.33× on Llama-3.2-3B and 2.44× on Mistral-7B-v0.3 over vLLM, while StepTrim with learned policy achieves 1.14× and 1.42×, respectively. Combining both components increases

throughput to 2.56× and 3.02×. The combined speedups should be interpreted together with the task accuracies in Table 1 (LoRazor-P50). StepTrim can provide additional savings even when GridTrim's row bound is already at its smallest bucket. Once all scheduled requests assigned to an adapter have released it, that adapter requires no further computation in the batch. If the active adapter count crosses an adapter-count specialization boundary, the engine can also select a graph with fewer adapter slots. Thus, release can reduce execution cost without further shrinking the row bound.

5.1.1 *GridTrim deepdive*

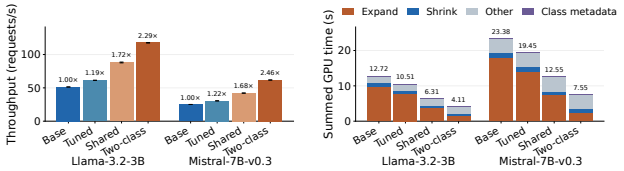

Figure 9. Left: throughput with tile tuning, shared row bounds, and two-class expand added progressively. Right: summed GPU execution time over the first 128 scheduling iterations.

We further decompose GridTrim's gains in Figure 9, with StepTrim disabled. Tile tuning alone improves throughput by 1.19× on Llama and 1.22× on Mistral over the baseline. Holding these tile settings fixed, shared-row bounds increase throughput by another 43.7% and 38.1%, while two-class expand adds a further 33.5% and 46.5%, respectively. Overall, two-class GridTrim achieves 1.92× and 2.02× the throughput of tuned full-batch execution. The GPU-time breakdown over the first 128 scheduling iterations shows that class separation primarily reduces expand time, while shrink and other GPU work remain largely unchanged. This is expected as the two-class partition is applied during the expand stage.

5.1.2 *StepTrim deepdive: Learned or Profiled?*

Table 4. Test accuracy and decode LoRA skipping for policies selected under a 2 percentage point validation accuracy-loss budget.

		Accuracy / Skip (%)	
Model	Task	LoRazor-Training-free	LoRazor
Llama	ScienceQA	87.60 / 25.46	**88.80 / 56.96**
	Spider	62.86 / 73.94	**62.96 / 76.27**
	MedQA-4	**53.02 / 85.23**	52.40 / 77.22
	GSM8K	43.52 / 30.56	41.24 / 43.82
Mistral	ScienceQA	91.60 / 48.62	91.60 / 32.96
	Spider	64.22 / 57.73	**64.60 / 96.77**
	MedQA-4	59.78 / 94.15	**60.17 / 98.53**
	GSM8K	53.98 / 43.59	53.90 / 47.64

We compare LoRazor with LoRazor-Training-free on each backbone-task pair. Both use GRIDTRIM and are selected under a 2-percentage-point validation accuracy loss budget relative to full-LoRA execution. The selected policies are frozen before test evaluation. The budget applies to validation accuracy, realized test accuracy may not match. Table 4 compares the two policies across all eight backbone–task pairs. LoRazor achieves higher observed accuracy and decode skipping on four pairs. The largest skipping gains occur on Llama-ScienceQA and Mistral-Spider, where skipping increases by 31.50 and 39.04 percentage points, respectively, alongside accuracy improvements of 1.20 and 0.38 points. Gains are smaller on Llama-Spider and Mistral-MedQA-4. However, LoRazor-Training-free achieves higher accuracy and skipping on Llama-MedQA-4 and higher skipping at equal accuracy on Mistral-ScienceQA. Both GSM8K pairs exhibit accuracy-skipping trade-offs. These results show that the benefits of learned policy are task-dependent and that profile-based release remains competitive. Appendix A.7 examines request-level release-cutoff distributions to help explain these differences.

5.2 Traffic distribution and row-size classes

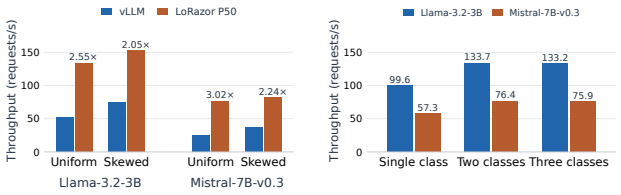

Figure 10. Effects of traffic distribution (left) and row-size classes (right) on throughput with 128 logical adapters.

Figure 10 examines the effects of adapter popularity and class-separated execution with 128 logical adapters. Uniform traffic distributes requests evenly across adapters over the workload, whereas skewed traffic concentrates requests on a subset of adapters. Under skewed traffic, 24 of the 128 logical adapters receive approximately 80% of requests. Appendix A.2 details the workload construction. LoRazor achieves speedups over vLLM of 2.55× and 3.02× under uniform traffic on Llama-3.2-3B and Mistral-7B-v0.3, respectively, compared with 2.05× and 2.24× under skewed traffic. Both systems achieve higher absolute throughput under skewed traffic. The smaller relative speedup reflects vLLM's improvement as well. Concentrating requests on popular adapters can improve per-adapter batching and reduce the number of active adapters within individual batches. For LoRazor, class-separated execution also limits the impact of large adapter groups on smaller groups' launch grids. The right panel isolates the effect of row-size classes by holding tile settings fixed and leaving shrink unchanged. Separating expand into two classes increases throughput from 99.6 to 133.7 requests/s on Llama and from 57.3 to

76.4 requests/s on Mistral, improvements of approximately 34% and 33%. Adding a third class ($64 \geq m_a > 16$) yields 133.2 and 75.9 requests/s, respectively, providing no further improvement in this experiment. We therefore use two classes in the main evaluation.

6 Related Work

6.1 Selective Adapter Execution

Prior work reduces adaptation cost by restricting where adapters operate or selecting which adapters contribute. AdapterDrop (Rücklé et al., 2021) reduces execution across model depth by removing adapters from lower Transformer layers, while ReFT (Wu et al., 2024b) learns interventions on selected hidden representations. PrAd (Ma et al., 2025) and PreFT (Lanpouthakoun et al., 2026) restrict adaptation to prompt tokens through specialized training, eliminating adapter computation during decoding. A complementary direction dynamically routes or combines adapters at the token level. Arrow (Ostapenko et al., 2024), X-LoRA (Buehler & Buehler, 2024), and LoRA Mixer (Li et al., 2026b) select or weight adapter contributions with routing applied across adapters, layers, or projection matrices. AdaFuse (Li et al., 2026a) further combines token-level pre-gating with fused execution to reduce routing and kernel-launch overhead. Beyond adapters, SkipDecode (Del Corro et al., 2023) and LayerSkip (Elhoushi et al., 2024) reduce backbone computation through layer skipping or early-exit mechanisms. LoRazor instead uses release policies to determine when requests can permanently switch from adapters to backbone-only decoding with limited impact on task accuracy.

6.2 Multi-LoRA Serving and Kernel Optimization

Multi-LoRA serving systems improve efficiency through heterogeneous batching, memory management, and scheduling. PetS (Zhou et al., 2022) supports heterogeneous parameter-efficient tasks through shared backbone operators and coordinated batching. FLoRA (Wen & Chaudhuri, 2024) allows each input example to use distinct low-rank adaptation weights. Punica (Chen et al., 2024) enables heterogeneous adapter batching through specialized kernels and consolidates workloads across GPUs. S-LoRA (Sheng et al., 2024) combines heterogeneous batching with unified paging of adapter weights and KV caches. dLoRA (Wu et al., 2024a) dynamically chooses merged and unmerged execution and coordinates request-adapter migration. Toppings (Li et al., 2025) overlaps adapter loading with CPU-assisted prefill. Chameleon (Iliakopoulou et al., 2025) combines adapter caching with adapter-aware scheduling. LoRAServe (Jaiswal et al., 2025) optimizes distributed adapter placement and routing. These systems improve where and how requested adapters execute. LoRazor on the other hand decides how long each request uses its adapter during the decode generation, and releases the adapter accordingly to reduce the adapter usage in each batch iteration.

6.3 Compact Adapter Representations

Prior work reduces the memory footprint required for task specialization. VeRA (Kopiczko et al., 2024) learns scaling vectors over shared frozen random projections. Compress then Serve (Gabrielsson et al., 2025) jointly compresses trained adapters into shared bases and adapter-specific scaling matrices. SALT (Li et al., 2026c) trains shared domain centroids and ultra-low-rank task residuals to reduce per-adapter storage and transfer costs. Beyond LoRA, DeltaZip (Yao et al., 2025) compresses weight differences between a shared backbone and fully fine-tuned models to support efficient concurrent serving. These methods change how adaptation is represented or stored to improve serving efficiency. LoRazor complements this direction by reducing adapters usage in decode generation and adapting the computation kernel to the remaining workload.

7 Discussion

The choice between learned and profiled release policy of LoRazor depends on the workload and backbone model. Although requests differ in their sensitivity to adapter release, a learned policy must identify favorable release points to exploit this variation. Currently, LoRazor improves both accuracy and skipping on some backbone-task pairs, while others favor profiled release or exhibit accuracy-skipping trade-offs. Future work could investigate signals that better distinguish requests permitting early release from those benefiting from longer adapter execution. Such signals could improve prediction or support hybrid policies that combine profiled timing with request-specific decisions. Furthermore, both release policies rely on validation-based calibration and provide no per-request correctness guarantee. Workload shifts may require recalibration. Selective adapter reactivation is another direction, but must account for the KV state accumulated during backbone-only decoding and the additional execution cost.

8 Conclusion

We present LoRazor, a multi-LoRA serving framework that enhances efficiency by strategically controlling the adapter execution duration during the decode stage. StepTrim employs release policies to permanently release adapters during decoding, boosting efficiency with marginal accuracy trade-offs. Meanwhile, GridTrim resizes kernel launches to match the remaining active adapter workload, eliminating unnecessary GPU block launches. Together, these techniques substantially improve multi-LoRA serving throughput with negligible impact on downstream task accuracy.

REFERENCES

Buehler, E. L. and Buehler, M. J. X-lora: Mixture of low-rank adapter experts, a flexible framework for large language models with applications in protein mechanics and molecular design. *APL Machine Learning*, 2(2):026119, 05 2024.

Chen, L., Ye, Z., Wu, Y., Zhuo, D., Ceze, L., and Krishnamurthy, A. Punica: Multi-tenant lora serving. *Proceedings of Machine Learning and Systems*, 6:1–13, 2024.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Del Corro, L., Del Giorno, A., Agarwal, S., Yu, B., Awadallah, A., and Mukherjee, S. Skipdecode: Autoregressive skip decoding with batching and caching for efficient llm inference. *arXiv preprint arXiv:2307.02628*, 2023.

Elhoushi, M., Shrivastava, A., Liskovich, D., Hosmer, B., Wasti, B., Lai, L., Mahmoud, A., Acun, B., Agarwal, S., Roman, A., et al. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12622–12642, 2024.

Gabrielsson, R. B., Zhu, J., Bhardwaj, O., Choshen, L., Greenewald, K., Yurochkin, M., and Solomon, J. Compress then serve: serving thousands of lora adapters with little overhead. In *Proceedings of the 42nd International Conference on Machine Learning*, ICML'25. JMLR.org, 2025.

Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Iliakopoulou, N., Stojkovic, J., Alverti, C., Xu, T., Franke, H., and Torrellas, J. Chameleon: Adaptive caching and scheduling for many-adapter llm inference environments. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, pp. 217–231, 2025.

Jaiswal, S., Arun, S., Parayil, A., Mallick, A., Mastorakis, S., Khare, A., Alverti, C., Amant, R. S., Bansal, C., Rühle, V., et al. Serving heterogeneous lora adapters in distributed llm inference systems. *arXiv preprint arXiv:2511.22880*, 2025.

Jin, D., Pan, E., Oufattole, N., Weng, W.-H., Fang, H., and Szolovits, P. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021. doi: 10.3390/app11146421.

Kong, X., Wu, J., Zhang, A., Sheng, L., Lin, H., Wang, X., and He, X. Customizing language models with instance-wise loRA for sequential recommendation. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Kopiczko, D., Blankevoort, T., and Asano, Y. Vera: Vector-based random matrix adaptation. In *International Conference on Learning Representations*, volume 2024, pp. 6815–6835, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Lanpouthakoun, A., Arora, A., Wu, Z., Pai, D., Keigwin, B., Jurafsky, D., and Potts, C. Preft: Prefill-only finetuning for efficient inference. *arXiv preprint arXiv:2605.14217*, 2026.

Li, Q., Kong, R., Li, Y., Cai, H., Wang, S., Kong, L., Chen, G., and Yin, D. Adafuse: Accelerating dynamic adapter inference via token-level pre-gating and fused kernel optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 31680–31688, 2026a.

Li, S., Lu, H., Wu, T., Yu, M., Weng, Q., Chen, X., Shan, Y., Yuan, B., and Wang, W. Toppings:{CPU-Assisted},{Rank-Aware} adapter serving for {LLM} inference. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pp. 613–629, 2025.

Li, W., Song, Z., Zhou, H., Yu, J., Zhang, Y., and Yang, W. Lora-mixer: Coordinate modular lora experts through serial attention routing. In Vondrick, C., Hariharan, B., Raffel, C., Pinto, L., Yang, D., and Faust, A. (eds.), *International Conference on Learning Representations*, volume 2026, pp. 14694–14716, 2026b.

Li, X., Wang, P., Wang, H., and Bagchi, S. Pin once, swap light: Subspace-aligned centroid-residual training for efficient ultra-lora serving. *arXiv preprint arXiv:2608.03579*, 2026c.

Li, X., Yan, N., and Mortazavi, M. S. Embodied task planning via graph-informed action generation with large language models. In *Forty-third International Conference on Machine Learning*, 2026d.

Lu, P., Mishra, S., Xia, T., Qiu, L., Chang, K.-W., Zhu, S.-C., Tafjord, O., Clark, P., and Kalyan, A. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *The 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

Ma, Y., He, J., and Fei, H. Prad: Prompt adaptive tuning for decoder-only language models. In *EMNLP (Findings)*, pp. 4729–4743, 2025.

Ostapenko, O., Su, Z., Ponti, E. M., Charlin, L., Le Roux, N., Caccia, L., and Sordoni, A. Towards modular llms by building and reusing a library of loras. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Rücklé, A., Geigle, G., Glockner, M., Beck, T., Pfeiffer, J., Reimers, N., and Gurevych, I. Adapterdrop: On the efficiency of adapters in transformers. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pp. 7930–7946, 2021.

Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., Gonzalez, J. E., and Stoica, I. Slora: Scalable serving of thousands of lora adapters. In Gibbons, P., Pekhimenko, G., and Sa, C. D. (eds.), *Proceedings of Machine Learning and Systems*, volume 6, pp. 296–311, 2024.

Wen, Y. and Chaudhuri, S. Batched low-rank adaptation of foundation models. In *International Conference on Learning Representations*, volume 2024, pp. 23423–23438, 2024.

Wu, B., Zhu, R., Zhang, Z., Sun, P., Liu, X., and Jin, X. dLoRA: Dynamically orchestrating requests and adapters for LoRA LLM serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 911–927, Santa Clara, CA, July 2024a. USENIX Association. ISBN 978-1-939133-40-3.

Wu, Z., Arora, A., Wang, Z., Geiger, A., Jurafsky, D., Manning, C. D., and Potts, C. ReFT: Representation finetuning for language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b.

Yao, X., Hu, Q., and Klimovic, A. Deltazip: Efficient serving of multiple full-model-tuned llms. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 110–127, 2025.

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., and Radev, D. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In Riloff, E., Chiang, D., Hockenmaier, J., and Tsujii, J. (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 3911–3921, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1425.

Zhang, Z., Rossi, R. A., Kveton, B., Shao, Y., Yang, D., Zamani, H., Dernoncourt, F., Barrow, J., Yu, T., Kim, S., et al. Personalization of large language models: A survey. *arXiv preprint arXiv:2411.00027*, 2024.

Zhou, Z., Wei, X., Zhang, J., and Sun, G. {PetS}: A unified framework for {Parameter-Efficient} transformers serving. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pp. 489–504, 2022.

A APPENDIX

A.1 Adapter execution during prefill

The limited accuracy loss observed for selective adapter release during decoding does not extend to skipping LoRA for the entire prefill. Figure 11 compares full-LoRA execution (LoRA enabled throughout prefill and decoding) with a configuration that disables LoRA only during prefill while keeping it enabled throughout decoding.

Removing LoRA from the entire prefill causes substantially larger downstream accuracy losses than many decode-release configurations. This experiment evaluates skipping the entire prefill; selectively skipping adapter execution within prefill remains unexplored.

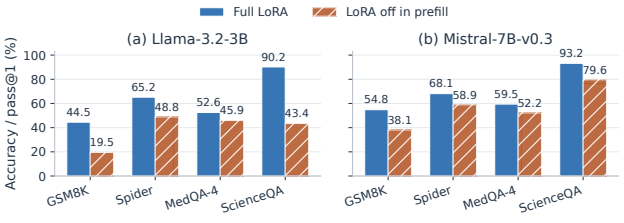

Figure 11. Disabling LoRA prefill significantly reduces task accuracy even when LoRA remains enabled throughout decoding.

Based on these results, LoRazor keeps adapters enabled throughout prefill and focuses on request-specific release during decoding. StepTrim decides when each request releases its adapter, while GridTrim efficiently executes the remaining active adapter rows.

A.2 Serving setup details

LoRA rank and target projections. For each backbone-task pair, we select the adapter with the highest validation Pass@1 accuracy among nine configurations spanning three target-module sets (Q/V, attention-only, and attention plus feed-forward projections) and three learning rates (5×10^{-5}, 10^{-4}, and 2×10^{-4}). All adapters use rank $r=8$. For Llama-3.2-3B, GSM8K, Spider, and ScienceQA adapt the attention projections $\{q_{\text{proj}}, k_{\text{proj}}, v_{\text{proj}}, o_{\text{proj}}\}$ and the feed-forward projections $\{\text{gate}_{\text{proj}}, \text{up}_{\text{proj}}, \text{down}_{\text{proj}}\}$; MedQA-4 adapts only the four attention projections. For Mistral-7B-v0.3, GSM8K, Spider, and ScienceQA adapt the four attention projections, whereas MedQA-4 adapts only q_{proj} and v_{proj}. Logical adapters replicate these task-specific weights under distinct adapter identities.

Skewed adapter traffic generation. The workload contains 32768 requests, equally divided among the four tasks. At 128 logical adapters, each task has 32 adapter identities. We randomly designate six identities per task as hot, obtained by rounding 0.2×32, and assign them 6554 of that task's 8192 requests, approximately 80%. The remaining 1638 requests are assigned to the other 26 identities. Thus, 24 of 128 adapters receive approximately 80% of the workload. Only adapter assignments change between uniform and skewed traffic; prompts and request orders remain unchanged.

Serving configuration and measurement. For vLLM and LoRazor, we allow at most 256 concurrently scheduled sequences and 2048 scheduled tokens per iteration, with chunked prefill enabled and prefix caching disabled. We reserve 32 GB for the KV cache and configure capacity for 128 resident adapters. We enable full CUDA graphs and adapter-count specialization, and disable asynchronous scheduling. Chunked prefill allows prompt chunks to share scheduling iterations with decoding requests, producing the heterogeneous adapter-group sizes targeted by two-class GridTrim.

For offline serving, we submit the complete request trace in a single generation call. Throughput is the number of completed requests divided by elapsed wall-clock time, including request admission and final draining but excluding initialization and warmup. Each measured run lasts at least 180 seconds. For open-loop serving, requests arrive according to a Poisson process at rates of 25-150 requests/s, independently of earlier completions. We report P90 TTFT and P90 TPOT in the open-loop serving along with the throughput.

A.3 Profile-based release configuration

Profile-based release applies LoRA during prefill and the first k decoding steps, then disables it for the remaining generation. For each backbone-task pair, we select the candidate k with the smallest validation accuracy loss among those achieving at least 50% decode LoRA skipping. The selected values are frozen before test evaluation. This criterion matches the P50 skipping target, but may not realize the exact skipping rates on the test split. The value of k used in the evaluation can be found in Table 5.

Table 5. Validation-selected release lengths k. A value of zero denotes prefill-only execution.

Task	Llama-3.2-3B	Mistral-7B-v0.3
GSM8K	32	0
Spider	8	16
MedQA-4	4	4
ScienceQA	16	16

A.4 Realized test-time skipping rates

P50 denotes a validation-time skipping target rather than a fixed test-time skipping rate. We select a threshold on

validation data and freeze it before test evaluation. Table 6 reports the resulting fraction of decode LoRA skipped.

Table 6. Test-time decode LoRA skipping rates (%) for LoRazor-P50.

Task	Llama-3.2-3B	Mistral-7B-v0.3
GSM8K	64.15	58.39
Spider	69.48	60.68
MedQA-4	54.53	82.73
ScienceQA	50.65	55.95

Realized skipping varies across tasks because the selection criterion requires at least 50% skipping on validation data, rather than exactly 50%, and the selected policy can behave differently on the test split. These rates are weighted by decode length: longer responses contribute more calls to the denominator. They therefore do not directly describe the fraction of requests released or their release positions. Our saved evaluation logs contain aggregate skipping counters but not per-request release steps, so release-position distributions are not reported.

A.5 Predictor training cost

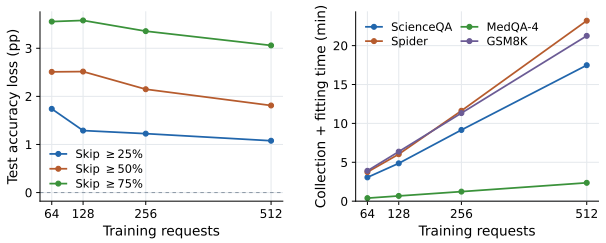

Figure 12. Predictor quality changes modestly with additional training requests, while the one-time offline collection and fitting remain inexpensive.

Figure 12 shows how predictor quality and training cost scale with the number of training requests. In the left panel, each curve corresponds to a validation skipping target of at least 25%, 50%, or 75% of decode LoRA calls, with a threshold selected separately at each training-set size. The vertical axis reports downstream test-accuracy loss. Increasing the training set from 128 to 512 requests yields only modest accuracy improvements (less than one percentage point). The right panel shows that data and label collection plus predictor fitting time scales approximately linearly with training-set size. This is a one-time offline cost incurred when preparing a risk predictor for a new adapter. The wall-clock cost varies across tasks because label collection generates paired continuations: longer outputs require more decoding, whereas MedQA-4 is substantially faster because its responses are shorter.

A.6 Releasing threshold selection

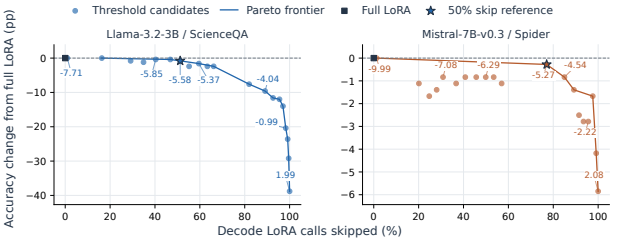

Figure 13. Release-threshold trade-offs Pareto frontier on validation data. Annotations show thresholds τ. Stars mark the selected operating points achieving at least 50% skipping.

We vary the release threshold τ while holding the predictor checkpoint and validation workload fixed. Candidate thresholds are defined by quantiles of the predictor's scores on training data, covering conservative to aggressive release settings; each candidate is then evaluated on the same validation workload. Figure 13 shows that more permissive thresholds increase decode skipping but can reduce accuracy. For Llama-3.2-3B on ScienceQA, a threshold of -5.58 skips 51.2% of decode LoRA calls with only 0.8 percent point accuracy loss; more aggressive thresholds incur substantially larger losses. For Mistral-7B-v0.3 on Spider, a threshold of -5.27 enables 77.1% skipping with only 0.3 percent point loss. This demonstrates performance sensitivity to the release threshold and supports selecting an operating point separately for each adapter. For the reported throughput experiments on P50, we select the threshold with the smallest validation accuracy loss among Pareto frontier candidates achieving at least 50% decode LoRA skipping on the validation data. We freeze this threshold before evaluating throughput on workloads constructed from the test data.

A.7 Profile-based or learned released policy?

To understand the task-dependent benefits of learned release policy , we examine how release sensitivity varies across requests. This analysis complements the calibrated policy comparison in Table 4 by characterizing hindsight opportunities for request-specific release.

For each validation request, we first generate an answer with full-LoRA execution. We then repeat generation at every integer release cutoff k within the tested horizon, retaining LoRA during prefill and the first k decoding calls before permanently switching to backbone-only decoding. Thus, $k = 0$ corresponds to prefill-only LoRA. Each continuation retains the KV cache produced before release and is scored using the task's evaluator. We restrict this analysis to requests answered correctly by full LoRA. For each request, we define its *stable-safe cutoff* as the smallest tested cutoff at which the answer is correct and remains correct at

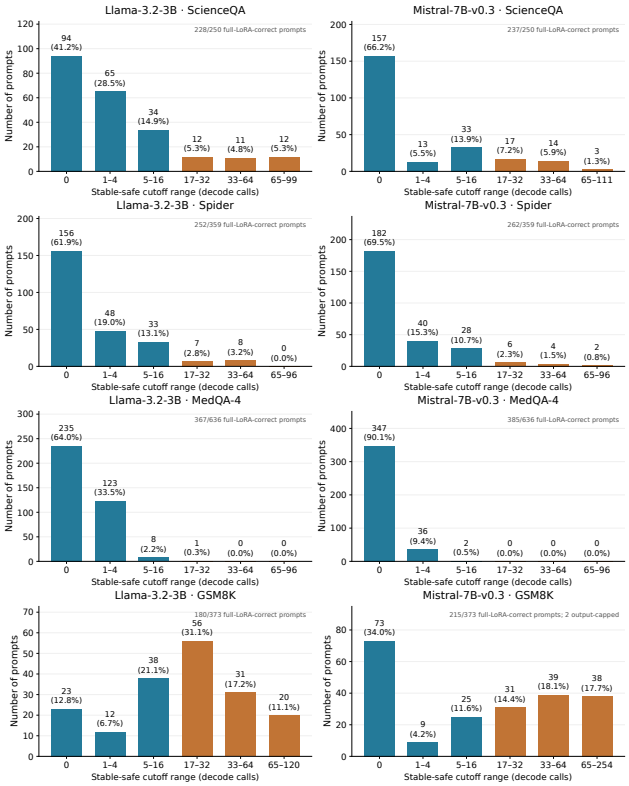

Figure 14. Distributions of stable-safe release cutoffs on validation requests answered correctly by full LoRA. A cutoff of k retains LoRA during prefill and the first k decoding calls.

every later tested cutoff. This definition accounts for non-monotonic release sensitivity: a request may be answered correctly at an early cutoff but incorrectly at a later one. Consequently, the stable-safe cutoff need not be the earliest cutoff that produces a correct answer.

Figure 14 groups requests by their stable-safe cutoffs. Percentages are calculated among full-LoRA-correct requests within each backbone-task pair.

On MedQA-4, stable-safe cutoffs are concentrated in the first few decoding calls: 97.5% of full-LoRA-correct Llama requests and 99.5% of Mistral requests have cutoffs at most four. This concentration helps explain why a profiled-based policy can be competitive, since there is relatively little variation in stable-safe cutoffs beyond the initial decoding steps. In Table 4, profile-based policy achieves both higher accuracy and higher skipping on Llama-MedQA-4. On Mistral-MedQA-4, profile-based policy already skips 94.15% of decode LoRA execution, leaving limited additional skipping headroom. Learned policy increases skipping by 4.38 percentage points and accuracy by 0.39 points.

ScienceQA and Spider contain substantial populations with immediate-release opportunities alongside requests with later stable-safe cutoffs. On Llama-ScienceQA, 41.2% of requests have cutoff zero, while 15.4% have cutoffs greater than 16. A shared profile-based policy must balance retaining adapters for later-cutoff requests against unnecessary execution for requests that permit earlier release. The simultaneous improvements of the learned policy in accuracy and skipping on Llama-ScienceQA are consistent with exploiting this variation.

Spider also contains large immediate-release populations: 61.9% of Llama requests and 69.5% of Mistral requests have cutoff zero. Although most remaining requests reach their stable-safe cutoff within 16 calls, a minority have later cutoffs. Learned policy improves both accuracy and skipping on both Spider pairs, with a substantially larger skipping improvement on Mistral.

GSM8K exhibits substantially later stable-safe cutoffs: 59.4% of full-LoRA Llama requests and 50.2% of Mistral requests have cutoffs greater than 16. The learned policy yields higher skipping but lower observed accuracy, with a 2.28-percentage-point accuracy decrease on Llama and a 0.08-point decrease on Mistral. These results illustrate accuracy-skipping trade-offs rather than simultaneous gains.